

Cybersecurity

HTTP with Sessions and Cookies

Lesson 13.1



Learning Objectives

By the end of today's class, you will be able to:

- 1 Understand HTTP requests and responses.
- 2 Use the `curl` command-line tool to make GET and POST requests and examine the responses.
- 3 Explain different cookie types (permanent and third-party).
- 4 Manage cookies using the Chrome extension Cookie-Editor.
- 5 Use Chrome's Developer Tools to audit HTTP request and response headers.



Welcome

The Web: A Security Perspective

In previous units, we learned about system administration and networking concepts.

This week, we'll build on those concepts as we explore how web-based communication works in the client-server model.

We'll examine how we communicate over the web (**networking**) and how websites and web applications are created and deployed (**system administration**).



Web Development Overview

Throughout this week, we will consider how web applications work from the perspectives of the user and the web developer/architect.

This week, we'll cover the following:

- What happens behind the scenes when we visit a webpage.
- How websites “remember” us through cookies and sessions.
- The architecture of web applications.
- Deploying and managing data in a web application.



Overview of Today's Topics

Today's topics will focus on what happens when we do everyday online activities, such as visit a website or log into an e-commerce site.

Specifically, we will look at:

- 1 How our browsers send and receive data from a web server.
- 2 Using our browser to dive deeper into the data being sent back and forth.
- 3 Using the command line to craft, manipulate, and examine data over the web.
- 4 How web applications “remember” who we are.



Intro to the **Modern Web**

Introduction to the Modern Web

It's increasingly important for security professionals to understand the web for the following reasons:

Organizations are incorporating more web-based applications and services, such as Salesforce, into their operations.

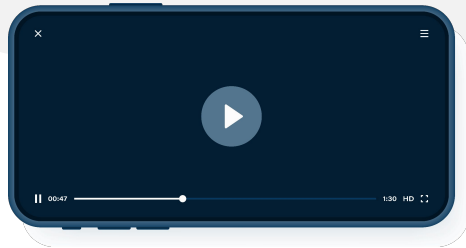
The Salesforce logo, which consists of a blue cloud shape with the word "salesforce" in white lowercase letters inside it.

salesforce

Companies are moving their on-premises infrastructure and services to the cloud—e.g., using cloud-based backup services for servers.



Most people access the web on a daily basis, whether for shopping or streaming videos.



Introduction to the Modern Web

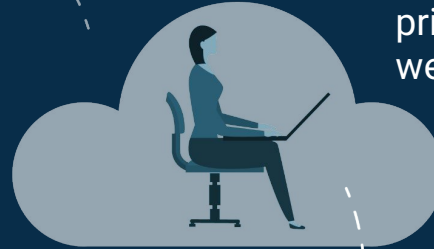
Some of the major security roles that focus on web technologies are:

Web application security engineers

must deeply understand how their company's web application works and how to secure it.



Penetration testers are ethical hackers who find and exploit vulnerabilities to gain privileged access to web applications.



Risk assessors

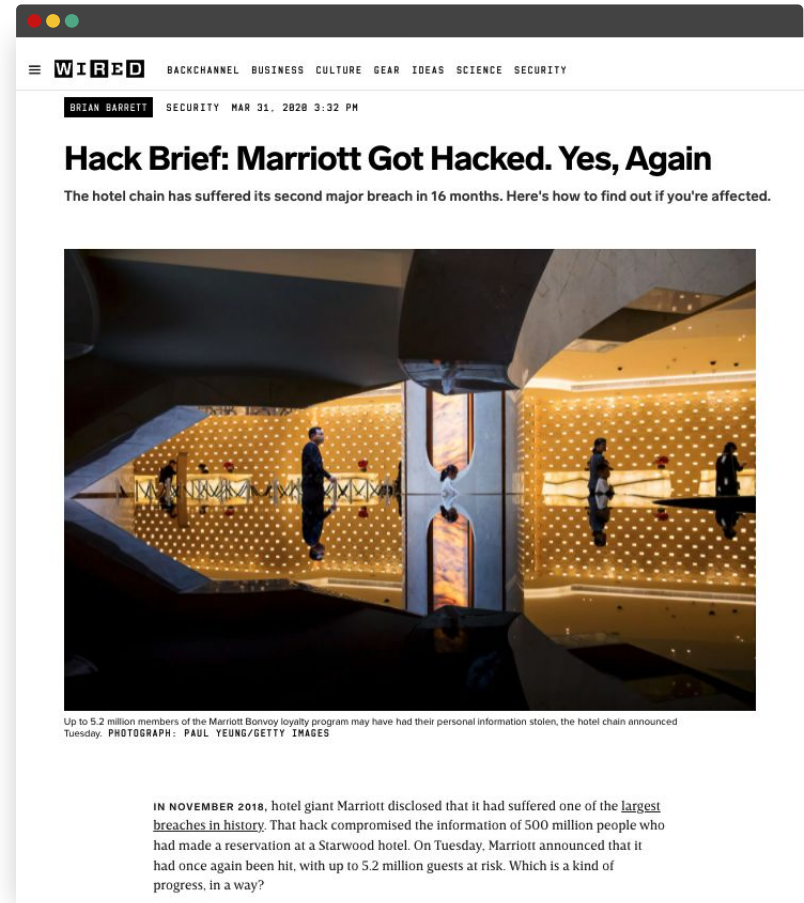
must understand the risk profiles and tolerances within web applications so they can help companies follow best practices and policies.



The Web: A Security Perspective

The threats embedded in the modern web are constantly growing.

- The web represents most organizations' largest attack surface.
- The number of users accessing the web at all times, through PCs, phones, and other devices, is growing constantly.
- The collective amount of personal data stored on the web is growing significantly.
- Attackers can steal and sell this data.



The image is a screenshot of a web browser displaying a Wired article. The browser's address bar shows the Wired logo and navigation links: BACKCHANNEL, BUSINESS, CULTURE, GEAR, IDEAS, SCIENCE, and SECURITY. The article's byline reads 'BRIAN BARRETT SECURITY MAR 31, 2020 3:32 PM'. The main headline is 'Hack Brief: Marriott Got Hacked. Yes, Again', followed by a sub-headline: 'The hotel chain has suffered its second major breach in 16 months. Here's how to find out if you're affected.' Below the text is a photograph of a modern hotel interior with a curved, illuminated wall and people walking. A caption below the photo states: 'Up to 5.2 million members of the Marriott Bonvoy loyalty program may have had their personal information stolen, the hotel chain announced Tuesday. PHOTOGRAPH: PAUL YEUNG/GETTY IMAGES'. At the bottom of the article, a paragraph begins: 'IN NOVEMBER 2018, hotel giant Marriott disclosed that it had suffered one of the largest breaches in history. That hack compromised the information of 500 million people who had made a reservation at a Starwood hotel. On Tuesday, Marriott announced that it had once again been hit, with up to 5.2 million guests at risk. Which is a kind of progress, in a way?'.

Wired

BACKCHANNEL BUSINESS CULTURE GEAR IDEAS SCIENCE SECURITY

BRIAN BARRETT SECURITY MAR 31, 2020 3:32 PM

Hack Brief: Marriott Got Hacked. Yes, Again

The hotel chain has suffered its second major breach in 16 months. Here's how to find out if you're affected.



Up to 5.2 million members of the Marriott Bonvoy loyalty program may have had their personal information stolen, the hotel chain announced Tuesday. PHOTOGRAPH: PAUL YEUNG/GETTY IMAGES

IN NOVEMBER 2018, hotel giant Marriott disclosed that it had suffered one of the largest breaches in history. That hack compromised the information of 500 million people who had made a reservation at a Starwood hotel. On Tuesday, Marriott announced that it had once again been hit, with up to 5.2 million guests at risk. Which is a kind of progress, in a way?



HTTP Requests and Responses



Preliminary Information

Securing the web requires in-depth knowledge of topics we've covered:



Client–Server Architecture



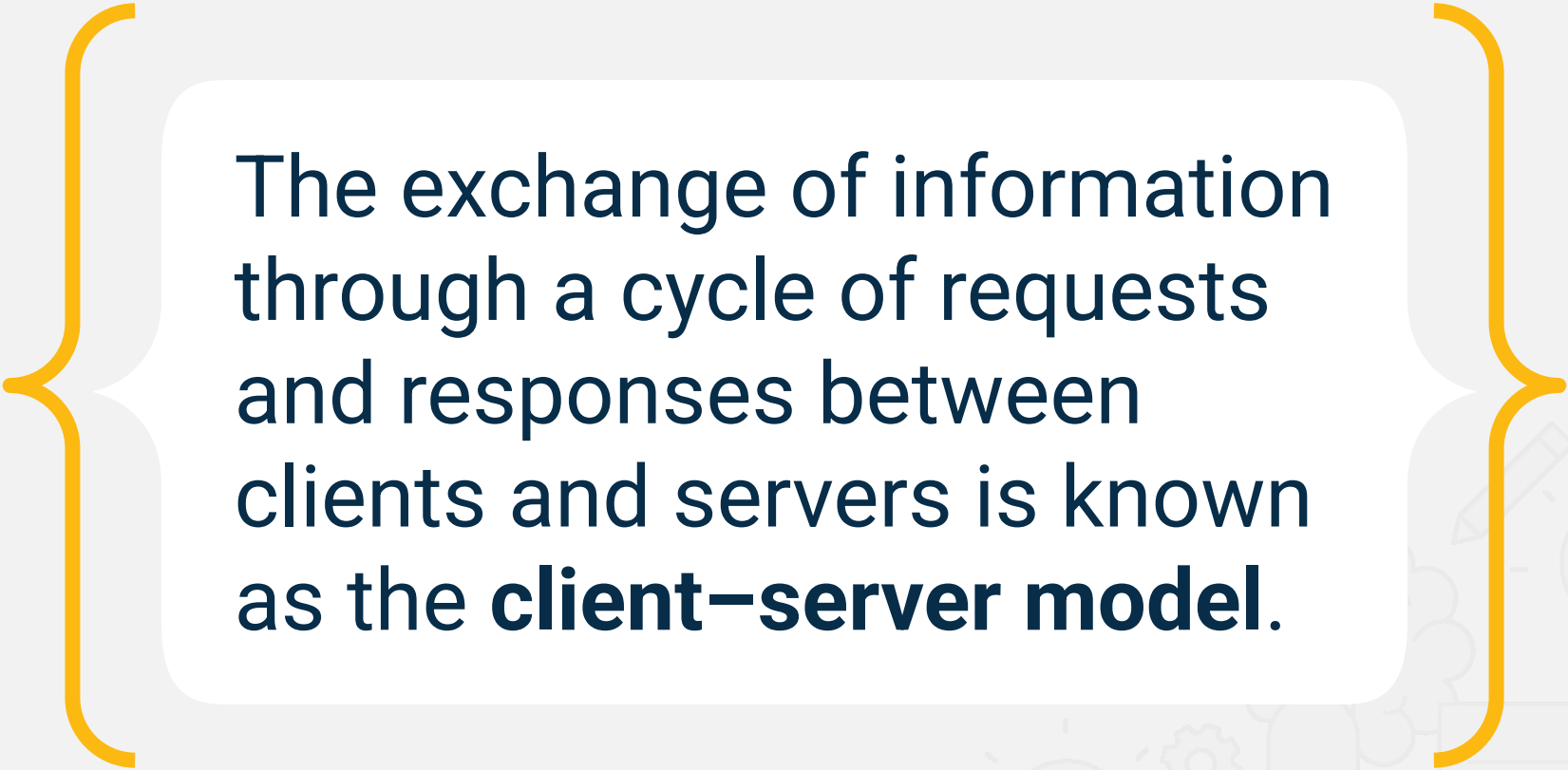
HTTP (Hypertext Transfer Protocol)



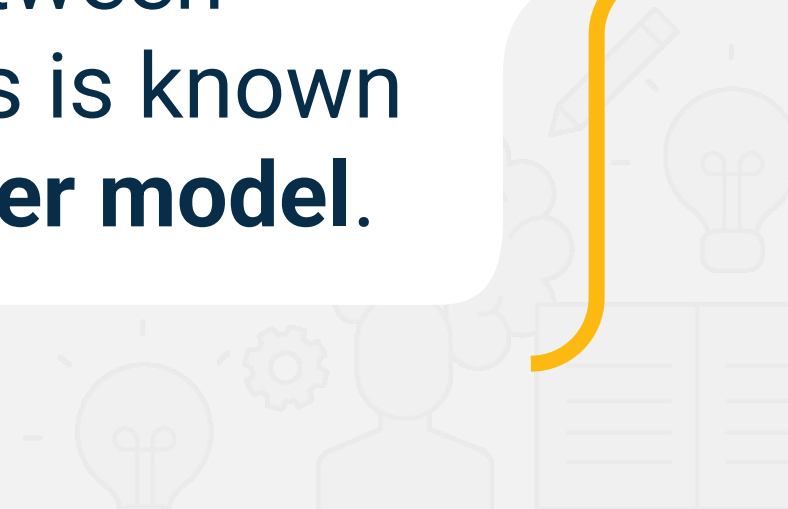
HTTP Requests



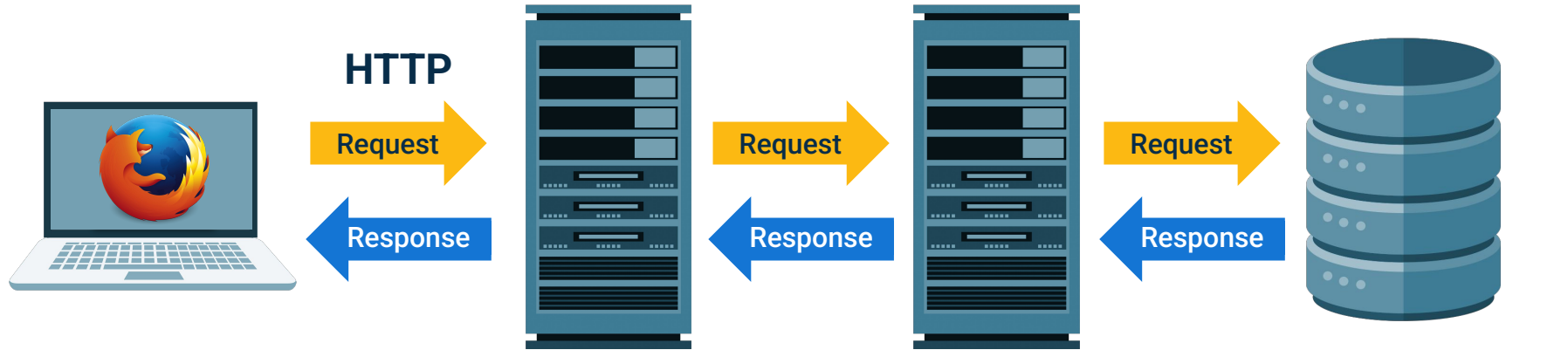
HTTP Responses



The exchange of information through a cycle of requests and responses between clients and servers is known as the **client-server model**.



Client–Server Architecture



Client

A user's device communicates with a server to request resources from it.

Web server

The server queries the resources from internal servers to which it is connected.

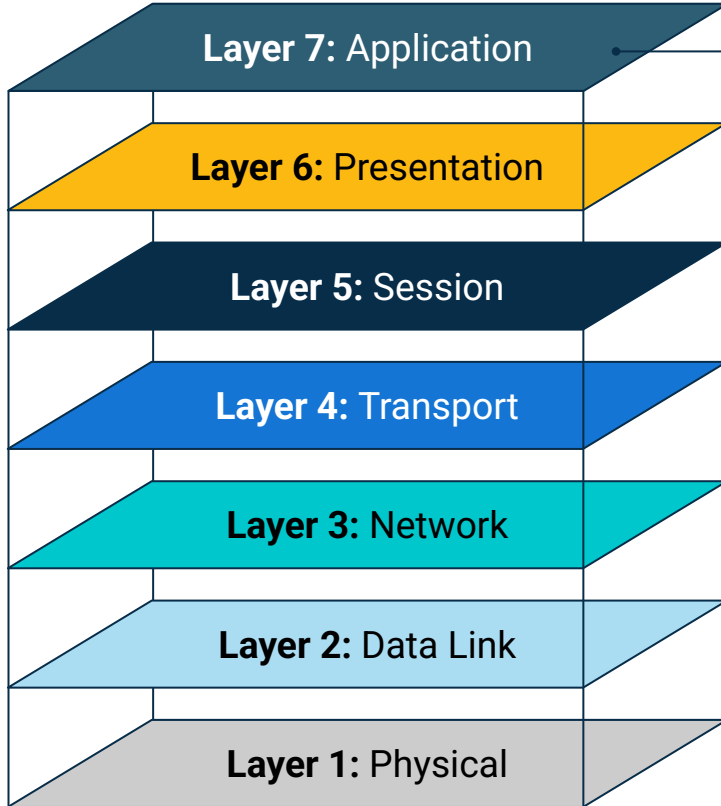
Application server

The application server sends and receives data from the database server.

Database server

The data resides within the database server.

HTTP Requests

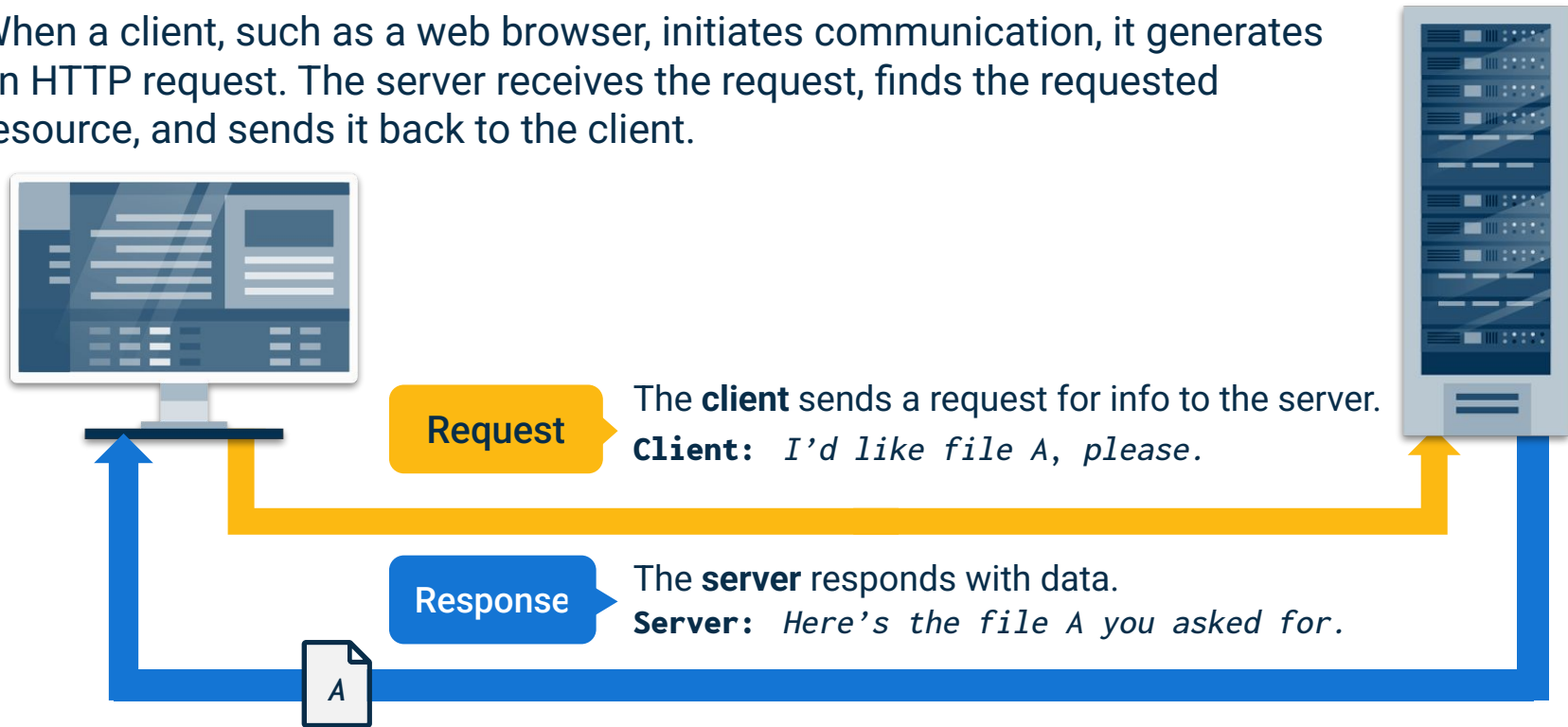


HTTP is an OSI **Layer 7** protocol that websites and web applications use to communicate with user devices.

HTTP Requests

HTTP is used to transfer web pages; static assets, such as images and HTML/markup files; and raw data, such as MP4 video or MP3 audio.

When a client, such as a web browser, initiates communication, it generates an HTTP request. The server receives the request, finds the requested resource, and sends it back to the client.



HTTP Methods

There are various types of requests, known as **HTTP methods**, that indicate the specific actions between the client and server. For example:

A user can request data
from a server.



*"I'd like to see my friend's
photos on Facebook."*



A user can give data
to a server.



*"Here are my credentials
for my LinkedIn account."*



A user can update data
on a server.



*"I'd like to reset my
Zoom password."*



HTTP Request Methods

There are various HTTP request methods:

HTTP Method	Description
GET	Requests data from a server.
HEAD	Identical to GET, but the server does not send the response body.
POST	Sends data to a source, often changing or updating a server.
PUT	Replaces current data with the new value.
DELETE	Deletes a specified resource.
CONNECT	Establishes a tunnel to the server.
OPTIONS	Lists the communication options for a target resource.



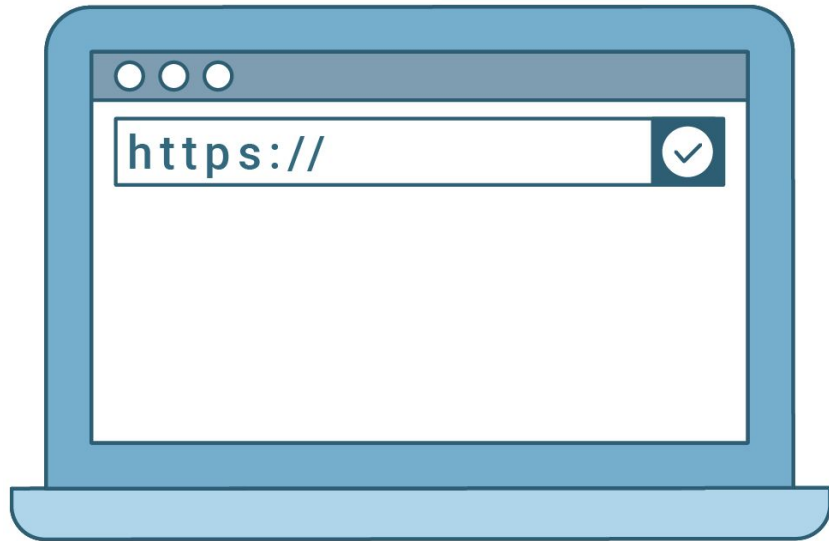
HTTP Requests and Responses

HTTP Request Methods

GET Requests

Requests data from a server.

When you open a browser and go to amazon.com, the HTTP client (your browser) asks to GET the data that the URL (amazon.com) points to. That data is the webpage.





HTTP Requests and Responses

HTTP Request Methods



POST Requests

Sends data to a source, often changing or updating a server. Once your browser goes to amazon.com, you log in to your Amazon account. The client sends a POST request that contains your credentials for logging in.



Anatomy of **HTTP Requests** and **Responses**

Anatomy of a GET Request

GET /js/analytics.js HTTP/1.1

Host: www.target-server.com

Connection: keep-alive

Upgrade-Insecure-Request: 1

Accept: text/js, text/html, */*

Accept-Language: en-us

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0

Request line

Headers

Whitespace

Request body (*optional*)

Anatomy of a GET Request

GET /js/analytics.js HTTP/1.1

Host: www.target-server.com

Connection: keep-alive

Upgrade-Insecure-Request: 1

Accept: text/js, text/html, */*

Accept-Language: en-us

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0

Request line

- Contains the request method, the name of the requested resource, and the version of HTTP in use.
- Can also contain query parameters, which the client can use to send data to the server.

Headers

Whitespace

Request body (optional)

Anatomy of a GET Request

GET /js/analytics.js HTTP/1.1

Host: www.target-server.com

Connection: keep-alive

Upgrade-Insecure-Request: 1

Accept: text/js, text/html, */*

Accept-Language: en-us

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0

Request line

Headers

- Contain additional details about the requested resource.
- Contain many actions with security implications, like authentication and remembering user resources.
- You don't have to remember every type of header. We'll cover some of the most important ones in class.

Whitespace

Request body (optional)

Anatomy of a GET Request

```
GET /js/analytics.js HTTP/1.1
Host: www.target-server.com
Connection: keep-alive
Upgrade-Insecure-Request: 1
Accept: text/js, text/html, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
```

Request line

Headers

Whitespace

- A blank line indicating the end of the request.

Request body (optional)

HTTP Request: Request Line

```
GET /js/analytics.js HTTP/1.1
```

```
Host: www.target-server.com
```

```
Connection: keep-alive
```

```
Upgrade-Insecure-Request: 1
```

```
Accept: text/js, text/html, */*
```

```
Accept-Language: en-us
```

```
Accept-Encoding: gzip, deflate
```

```
User-Agent: Mozilla/4.0
```

The request method is GET.

The requested resource is `/js/analytics.js`.

This resource is the file path from the domain stated in the header.

The request browser uses HTTP version 1.1.

HTTP Request: Headers

```
GET /js/analytics.js HTTP/1.1
Host: www.target-server.com
Connection: keep-alive
Upgrade-Insecure-Request: 1
Accept: text/js, text/html, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
```

Host contains the domain name of the target server.

Note that the protocol (HTTP) + host + file path in the request line is the URL we see in the browser:

http://www.target-server.com/js/analytics.js

HTTP Request: Headers

```
GET /js/analytics.js HTTP/1.1
Host: www.target-server.com
Connection: keep-alive
Upgrade-Insecure-Request: 1
Accept: text/js, text/html, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
```

Connection instructs the server to keep the TCP connection open after sending the response. The TCP is the connection used for the HTTP transfer.

This allows the client server to reuse the connection for later requests.

The alternative is performing a TCP handshake, which is much slower.

HTTP Request: Headers

```
GET /js/analytics.js HTTP/1.1
Host: www.target-server.com
Connection: keep-alive
Upgrade-Insecure-Request: 1
Accept: text/js, text/html, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
```

Upgrade-Insecure-Requests instructs the server to turn this HTTP connection into HTTPS, so the response and future communications will be encrypted.

Accept informs the server that the client expects a JavaScript or HTML document in response, but will accept data of any type (*/*).

User-Agent informs the server that this request comes from a Mozilla 4.0 browser.

Headers

Other important headers include:

Authorization	Contains credentials to authenticate a user with a server.
Referer	Contains the address of the previous webpage that linked to the currently requested page. Allows servers to identify where people are visiting from and use that data for analytics, logging, or optimized caching. (If a link from a Google search led to the current page, the referer is Google.)
Cookie	Contains stored HTTP cookies previously sent by the server with the Set-Cookie response header.

Requests with Query Parameters

GET requests can also request data with query parameters:

```
GET /articles?tag=latest&author=jane HTTP/1.1
```

```
Host: www.target-server.com
```

```
Connection: keep-alive
```

```
Upgrade-Insecure-Request: 1
```

```
Accept: text/js, text/html, */*
```

```
Accept-Language: en-us
```

```
Accept-Encoding: gzip, deflate
```

```
User-Agent: Mozilla/4.0
```

Query parameters are useful for specifying the parts of a resource to receive or send data.

This **GET** request specifically requests articles with two Parameters: the **latest** articles, created by the author **jane**.

Requests with Query Parameters

GET requests can also request data with query parameters.

```
GET /articles?tag=latest&author=jane HTTP/1.1
```

Note the syntax for **parameters**:

```
[path]?[firstParam]=[value]&[secondParam]=[value]
```

Query parameters are useful for specifying the parts of a resource to receive or send data. This GET request specifically requests articles with two parameters: the **latest** articles, created by the author **jane**.

Anatomy of a POST Request

HTTP POST requests are similar to GET requests, but include a request body below the whitespace.

```
POST /api/users HTTP/1.1
Host: www.target-server.com
User-Agent: Mozilla/4.0
Accept: */*
Content-Length: 23
Content-Type:
application/x-www-form-urlencoded
```

```
username=Jane+Doe&password=p455
```

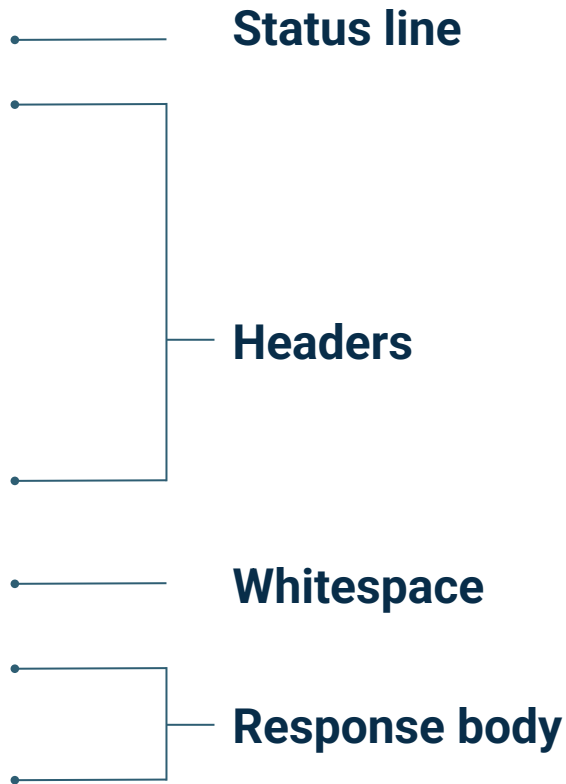
The request body may contain information such as login credentials or a file to be uploaded.

In the current example, our request body contains login credentials.

Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
Date: Nov 12 02:12:12 2018
Server: Apache/2.4.7 (Ubuntu)
X-powered-By: PHP/5.5.9-lubuntu4.21
Cache-Control: no-cache
Set-Cookie: SESSID=8toks; httponly
Content-Encoding: gzip
Content-Length: 698
```

```
function getStats(event) {
  ...
}
```



Anatomy of an HTTP Response

Status line	Contains the response status code and phrase that translates the result, such as OK or Error.
Headers	Contain additional information about the response, similar to request headers.
Whitespace	Separates the header from the following response body.
Response body	Contains the resource requested by the client.

Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
```

```
Date: Nov 12 02:12:12 2018
```

```
Server: Apache/2.4.7 (Ubuntu)
```

```
X-powered-By: PHP/5.5.9-1ubuntu4.21
```

```
Cache-Control: no-cache
```

```
Set-Cookie: SESSID=8toks; httponly
```

```
Content-Encoding: gzip
```

```
Content-Length: 698
```

```
function getStats(event) {  
  ...  
}
```

The response uses unencrypted HTTP/1.1.

The status code is **200 OK**, meaning that the request was processed properly.

Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
```

```
Date: Nov 12 02:12:12 2018
```

```
Server: Apache/2.4.7 (Ubuntu)
```

```
X-powered-By: PHP/5.5.9-lubuntu4.21
```

```
Cache-Control: no-cache
```

```
Set-Cookie: SESSID=8toks; httponly
```

```
Content-Encoding: gzip
```

```
Content-Length: 698
```

```
function getStats(event) {  
    ...  
}
```

Date contains a timestamp of when the response was generated.

Server indicates the server is running PHP version 5.5.9 on Ubuntu, with kernel version 4.21.

Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
Date: Nov 12 02:12:12 2018
Server: Apache/2.4.7 (Ubuntu)
X-powered-By: PHP/5.5.9-lubuntu4.21
Cache-Control: no-cache
Set-Cookie: SESSID=8toks; httponly
Content-Encoding: gzip
Content-Length: 698
```

```
function getStats(event) {
  ...
}
```

Set-Cookie instructs the client to create a cookie called **SESSID** with the value **8toks** and that this cookie can only be set by the server with HTTP.

We'll discuss cookies and **httponly** in greater detail later.

Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
Date: Nov 12 02:12:12 2018
Server: Apache/2.4.7 (Ubuntu)
X-powered-By: PHP/5.5.9-lubuntu4.21
Cache-Control: no-cache
Set-Cookie: SESSID=8toks; httponly
Content-Encoding: gzip
Content-Length: 698
```

```
function getStats(event) {
  ...
}
```

Below the whitespace is the response body, which contains the source code of the resource requested in the **GET** request.

Status Codes

General status codes include:

200 Codes

Indicate success.

300 Codes

Indicate multiple choices, meaning the server can respond to the request in more than one way.

400 Codes

Indicate client errors, meaning the client sent an improperly formatted request.

500 Codes

Indicate server errors, meaning the server application failed somehow.

**You probably
recognize the
404 error.**



404 ERROR

Sorry, the page not found



A standard reference site
for general headers
and web information is
Mozilla Developer Network.

Key Takeaways

- 1 HTTP requests are sent from an HTTP client to an HTTP server.
- 2 HTTP responses are sent back from the HTTP server as a response to the client.
- 3 HTTP requests include a request line, request header, and optional request body.
- 4 HTTP responses include a status line, response header, and usually, a response body.
- 5 Query parameters allow you to be specific about the parts of a resource you want to send or receive data from.

Key Takeaways

6 GET requests ask an HTTP server for resources, such as a whole webpage.

7 POST requests send data to an HTTP server's resources, such as login credentials or images for a webpage.

8 PUT requests also send data to an HTTP server, but are often used to overwrite resources, such as updating part of a webpage.

9 OPTIONS requests ask an HTTP server to respond with all HTTP methods that the HTTP server is programmed to respond to.



Activity:

HTTP Request and Response

In this activity, you will be playing the role of a web app security engineer.

- There is a breach in one of your company's HTTP servers. Before the server goes down, a suspicious IP address sends it a rapid sequence of requests and responses.
- You are tasked with investigating the remaining HTTP logs to figure out what happened during the attack on the web server.

Suggested Time:
15 Minutes

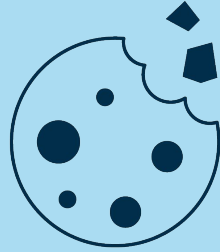


Time's up!
Let's review



Questions?

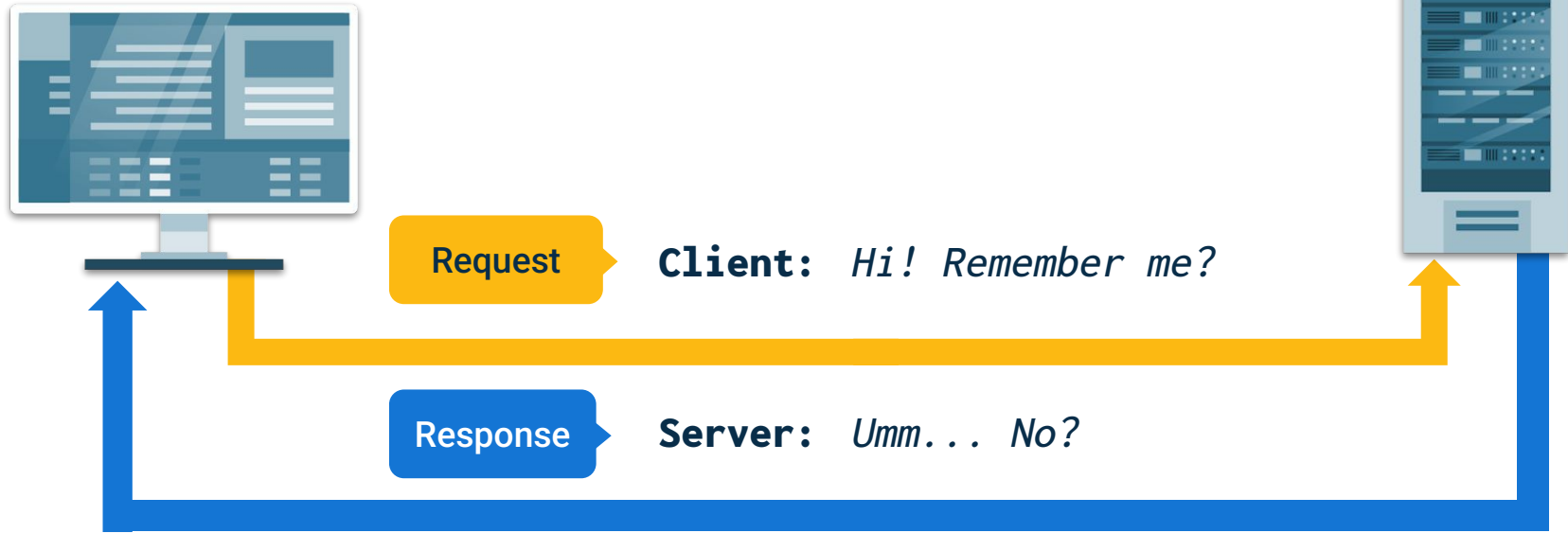




Sessions and Cookies

Sessions and Cookies

HTTP resources are inherently **stateless**, meaning that, whenever your browser requests a webpage, there's no way for that webpage to distinguish you from anyone else.



Sessions and Cookies

Websites need a way to deliver content that is specific to each user. To do so, they establish sessions with cookies.

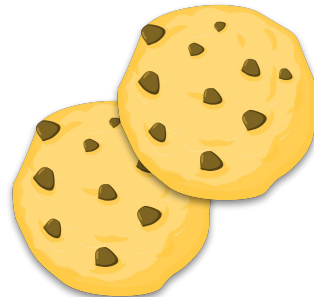
Sessions

Unique server-side sets of user data used to customize webpages for the specific user accessing them.



Cookies

Small pieces of text data that, when sent by an HTTP server's response header, are saved by the user's HTTP client.



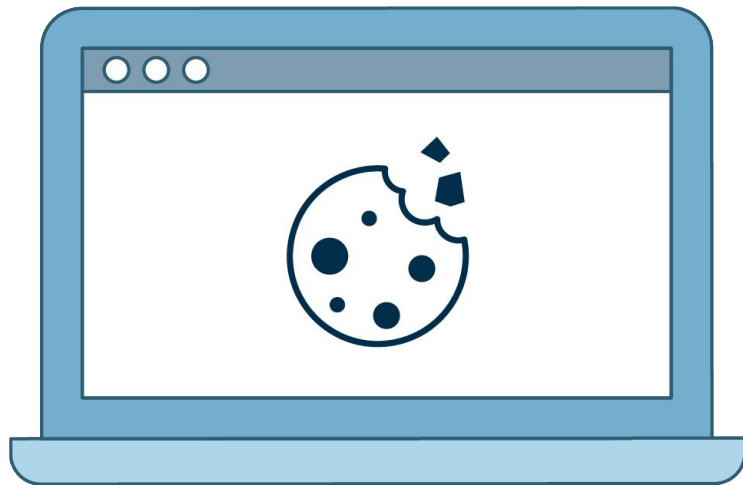
Sessions and Cookies

Cookies are used primarily for three things:

- Session Management
- Personalization
- Tracking

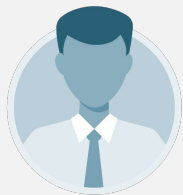
Common cookie types:

- Session Cookies
 - Generated when user interacts with a website
- Persistent Cookies
 - Stay on a user's computer for specified time
- Token Authentication Cookies
 - Help protect sensitive information
- Third-Party/Tracking Cookies
 - Record user activity
- Zombie Cookies
 - Used for malicious purposes



Sessions and Cookies

You visit a shopping website and register an account with your first and last name. As soon as you do this, a session is created that saves your name to the site's database.



User without cookies

GET /page.html HTTP/1.1
Site: example.com
Cookie: <None>



Default webpage



Users with cookies

GET /page.html HTTP/1.1
Site: example.com
Cookie: User=Bob



Default webpage

Cookie-detected
auto-forwarded
GET request



Personalized webpage

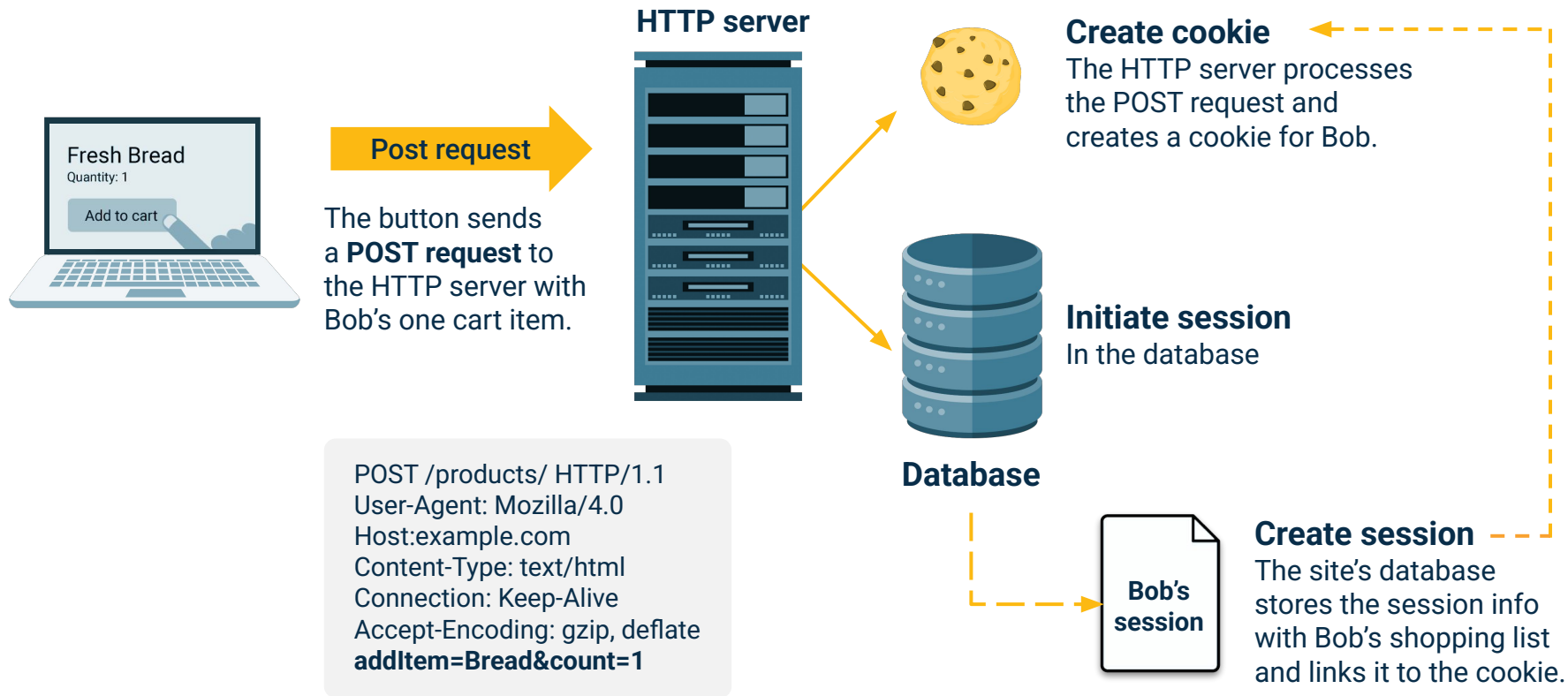
How a Session is Created

Before the session:

Bob visits a shopping site and clicks a button to add one loaf of bread to his shopping cart.



How a Session is Created



How a Session is Created

The HTTP server sends a response back to Bob's browser with:

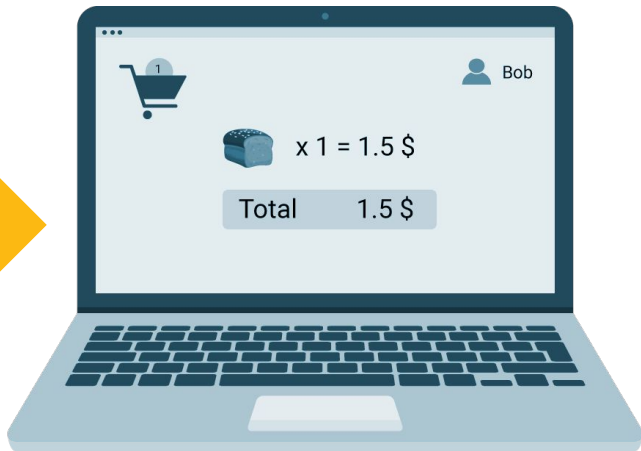
A **cookie**, sent through the **response header**.

```
HTTP/1.1 200 OK
Date: Fri, 13 Mar 2020 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Content-Length: 88
Content-Type: text/html
Set-Cookie: cart=Bob
<html><head>Bob's cart<head>...
```

Bob's **session information**, embedded in the **response body**.



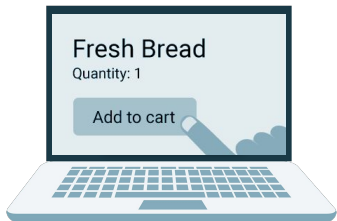
Response



How a Session is Created

Resuming the response:

When Bob revisits the webpage, his browser sends the cookie back through the **request header**.

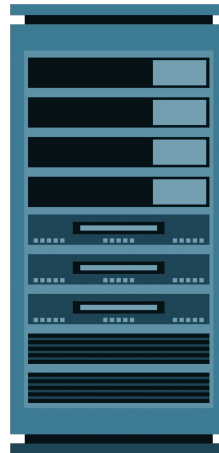


```
GET /stuff/ HTTP/ 1.1
User-Agent: Mozilla/4.0
Host: example.com
Cookie: cart=Bob
Content-Type: text/html
Connection: Keep-Alive
Accept-Encoding: gzip, deflate
```



Response

HTTP Server



Sessions and Cookies

Cookies and Attacks

Cookies can be set and read by **JavaScript**, a powerful web-based programming language that attackers often use in attacks such as:

- Cross-Site Scripting
- Clickjacking
- Session Hijacking

We'll cover these types of attacks in later modules.



How a Session is Created

Implementations of cookies vary between sites. Some are more secure than others.

1 If Bob's account was the fifth account to ever be created on a website, his cookie might look like **Set-Cookie: Session-ID=5** .

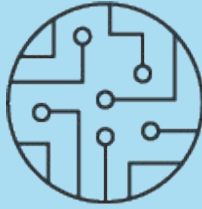
2 An attacker can change their cookie's value so that when they make a request, they do so as Bob.

3 Modern session cookies contain uniquely hashed values specific to users, e.g., **Set-Cookie: Session-ID=299367cd4c0e2c6d3ae6e09eda4e0590** .

4 While this keeps session ID cookies from being reverse-engineered, an attacker can still commit man-in-the-middle attacks by stealing cookies.

Summary

-  A cookie is used to implement a user's session.
-  A client and server exchange cookies through headers.
-  Clients save cookies locally.
-  A client sends a cookie via the cookie header.
-  A server sends a cookie via the Set-Cookie header.
-  Bad implementations of cookies can be spoofed or stolen, and cookies with modern implementations can be stolen.



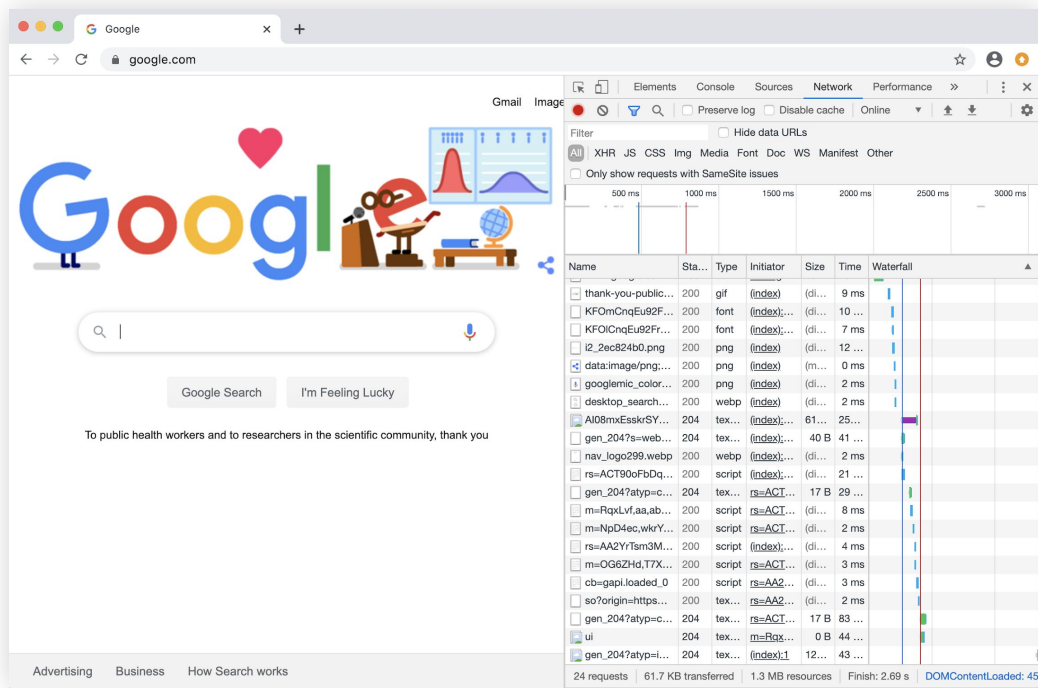
Examine **HTTP, Cookies, and Security**

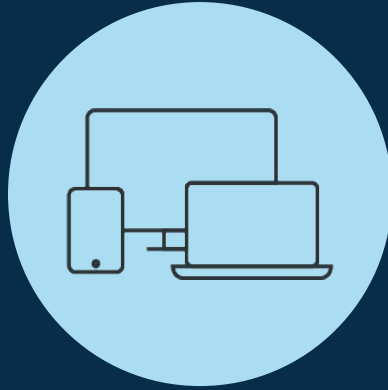
Chrome Developer Tools

Now, we'll deconstruct and examine a web application using the tools within a browser.

These allow us to:

- Understand the request and response flows between an HTTP client and server.
- Examine the security implementations of a website through its response headers.
- Identify where cookies are used in the HTTP request-response cycle.





Instructor Demonstration

Chrome Developer Tools



Activity:

Inspect with Developer Tools

For this activity, you will continue in your role as a web app security engineer.

- You are tasked with examining the various requests and response headers of one of your vendor's websites to assess its HTTP security.
- For this activity, you'll use Chrome Developer Tools to examine the initial request and response set from www.crowdstrike.com.

Suggested Time:
10 Minutes



Time's up!
Let's review

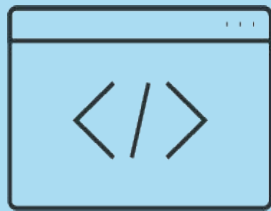


Questions?



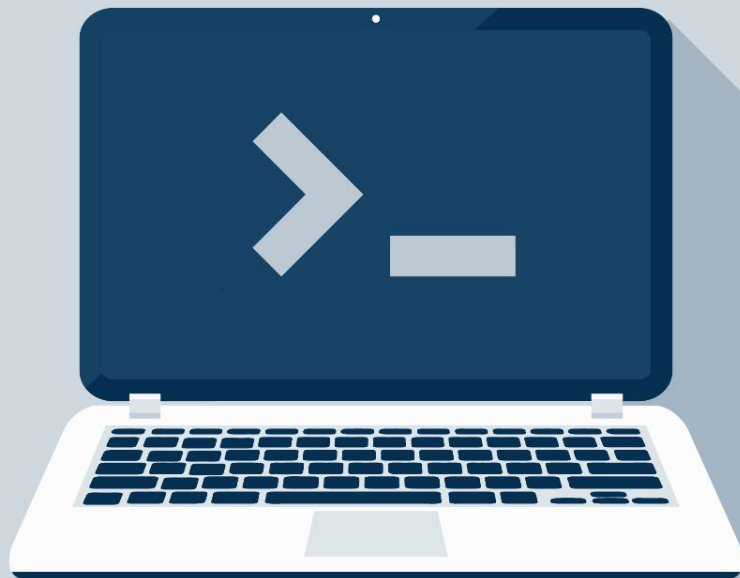


Break
15 mins



curl

It's not always possible to examine **HTTP requests and responses** through a browser. Sometimes the tools you can use to send and receive **HTTP requests** are limited.

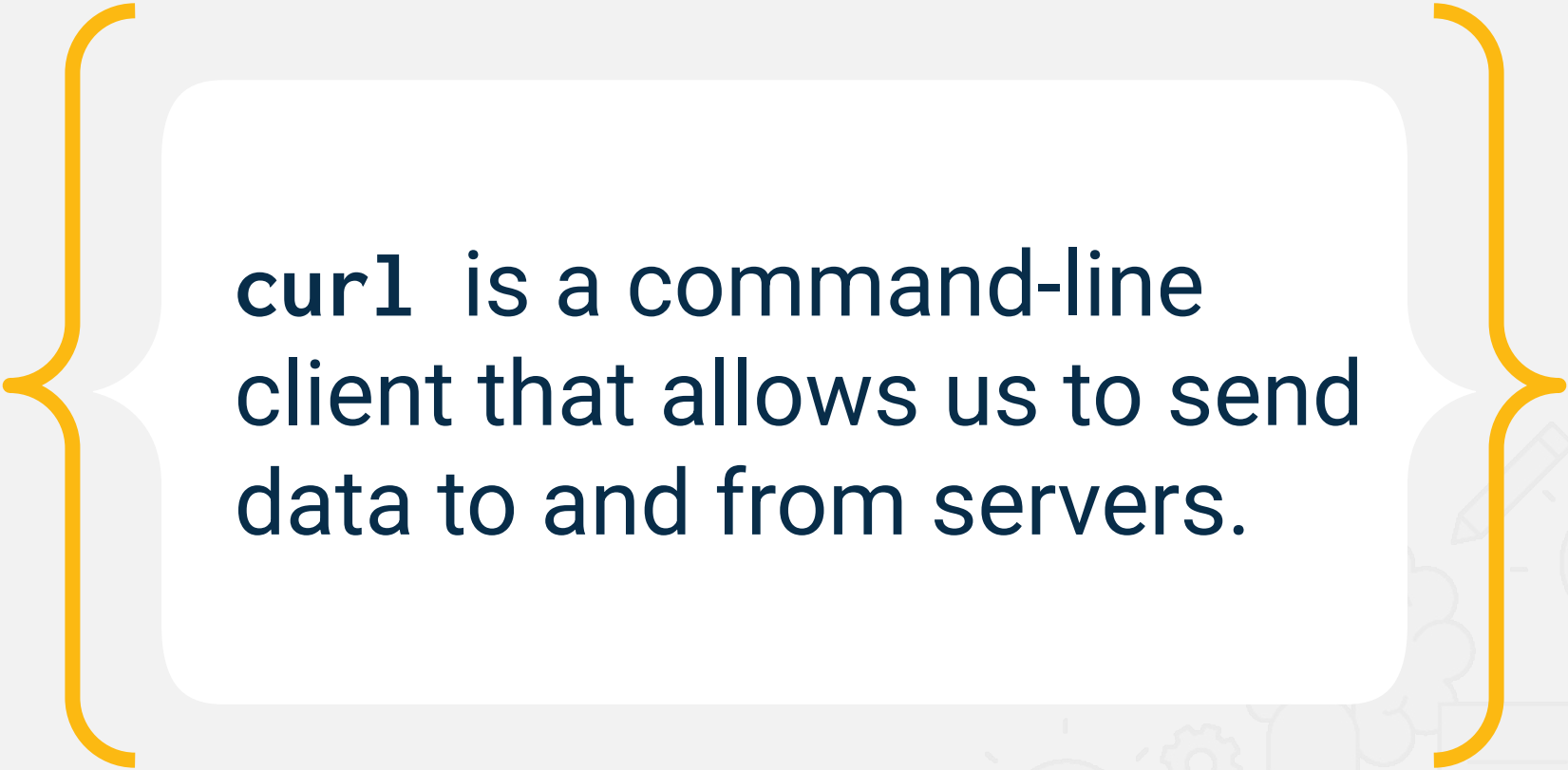


Introducing curl

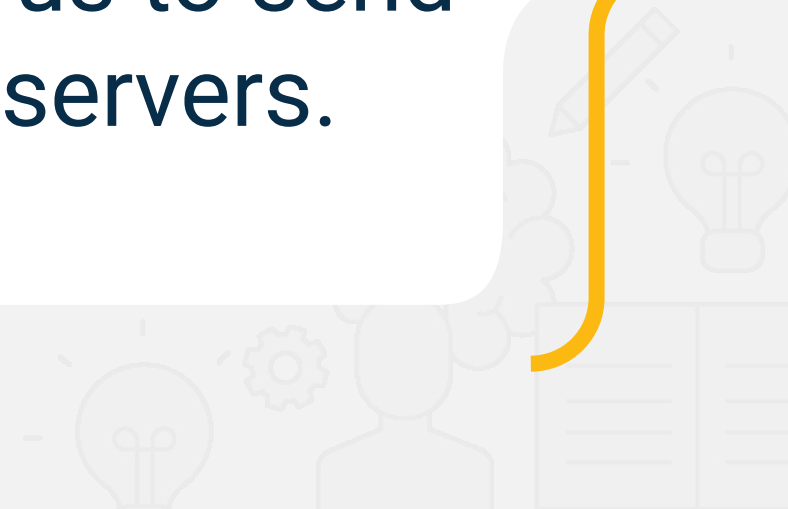
For example, when working through a container that has no user interface, you'll need a command-line tool to send and receive HTTP requests.

The command-line tool, **curl**, allows security professionals to quickly test HTTP requests in a way that can be automated, while allowing them to make adjustments as they work.





curl is a command-line client that allows us to send data to and from servers.

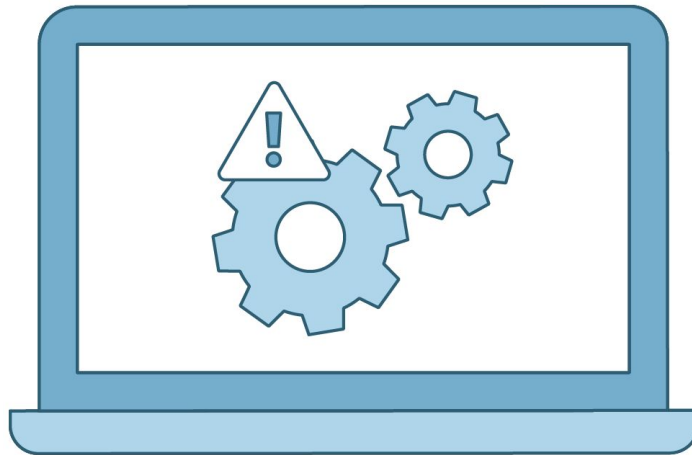


curl

Sends Customized HTTP Requests

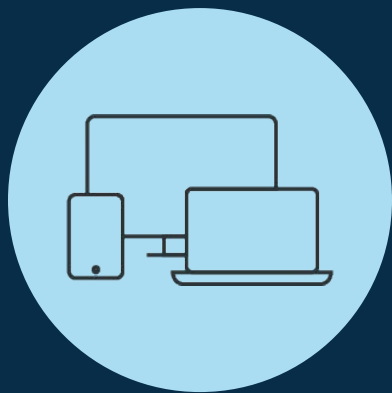
curl allows security professionals to:

- Test web server security configurations
- Ensure web servers don't leak sensitive data
- Verify servers only respond to certain requests
- Look for vulnerabilities on a web server





As a command-line tool, `curl` can be used in conjunction with other command-line tools, such as `grep`, `sed`, and `awk`.



Instructor Demonstration

`curl`



Activity:

Using `curl`

For this activity, you will continue in your role as a web app security engineer.

- You are tasked with writing various `curl` commands to interact with an HTTP web server.
- You will use various `curl` options, request methods, and request headers to return specific responses.

Suggested Time:
15 Minutes



Time's up!
Let's review



Questions?





We'll use a Chrome extension called **Cookie-Editor** to take a closer look at sessions and cookies in our browser.

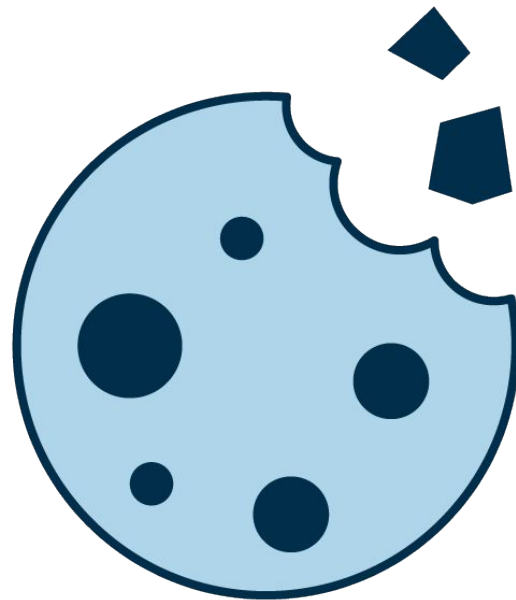


Cookie Editor

Managing User Sessions

Cookie-Editor is an extension that works with Google Chrome allowing:

- Import users' cookies
- Export users' cookies
- Save users' cookies

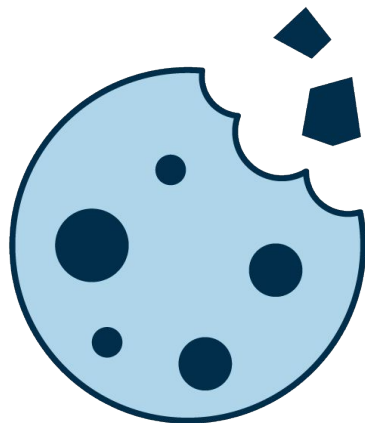


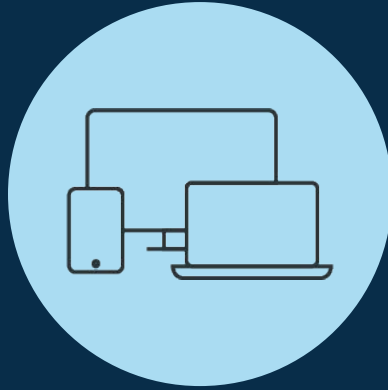


Demonstration Scenario

In this demo for using **Cookie-Editor**, we'll complete the following steps:

1. Enable the Chrome extension Cookie-Editor.
2. Create a user on a webpage and export their cookie to the clipboard with Cookie-Editor.
3. Create a new user, which will overwrite the old user's cookie and session.
4. Use the Cookie-Editor extension to import back the first user's session to show how a simple tool can be used to swap sessions.





Instructor Demonstration

Cookie-Editor



Activity:

Swap Sessions

For this activity, you will continue your role as a web app security engineer.

- After the previous web server was compromised, your company decides to use WordPress as a platform for its new e-commerce site.
- You are tasked with using the Chrome browser extension Cookie-Editor with your company's WordPress site to examine the process an attacker might take to hijack a session.

Suggested Time:
25 Minutes



Time's up!
Let's review



Questions?





Next class

Microservices and Web Application Architecture



Thank you